

MCP & Plugin Supply Chain Review Checklist

MCP = Model Context Protocol - vendor-neutral integration layer that lets AI coding agents (Claude Code, Codex, opencode, etc.) connect to external systems: Jira, Confluence, GitHub, databases, internal APIs. Think of MCP servers as the JDBC drivers of AI agents - without one, the agent is isolated from your stack.

One page. Apply before every new plugin or MCP server install - especially in regulated environments.

Why this matters (30-second framing)

A Claude Code plugin or MCP server is executable code that runs with your developer's permissions. It can read your repo, write files, call your APIs, post to Slack, query your databases. Treating it like any other third-party dependency is the only safe default.

Banking-specific: the EU AI Act regulatory clock is in motion. As of May 21, 2026, the Commission's May 7 Digital Omnibus agreement sets rules for certain high-risk standalone areas from 2 December 2027 and product-integrated AI systems from 2 August 2028, while transparency and other obligations still phase in earlier. Treat exact scope as a legal-check item, not legal advice from this workshop - but governance readiness is needed regardless of the precise enforcement calendar. Source: <https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai>

Pre-install vetting - 5 questions

Before installing ANY plugin or MCP server, answer these in writing (paste into the PR description that adopts it):

1. Source provenance

- Repo URL: _____
- First-party Anthropic? Or community? Maintainer name + GitHub history checked?
- Latest commit date (sanity: not abandoned, not suspiciously hyperactive)
- Star count + fork count (cross-reference against creation date for plausibility)
- Any security advisories or CVEs?

2. Permissions scope

- What tools does it request? (Read, Write, Bash, Network, MCP-specific)
- Does it ask for Bash or Network access? If yes, what for, specifically?
- Does it bundle any subagents? With what tool permissions?
- Does it install hooks? PreToolUse hooks can intercept ALL agent actions - high-risk surface.
- Does the MCP server need credentials? If yes - how are they stored? Env var -> can be exfiltrated. OS keychain or Vault -> acceptable. Plaintext in config file -> automatic reject.

3. Code review

- Skim the entry points (commands/, agents/, hooks/) - anything unusual?

- Any dynamic code execution (eval-style calls, subprocess invocation, shell escapes)?
- Any network calls to non-obvious domains (telemetry, "analytics", custom servers)?
- Any data exfiltration risk (reads secrets, sends prompts elsewhere)?

4. Maintenance and update cadence

- Release cadence - too slow (abandoned)? Too fast (unstable)?
- Open issue count and response times
- Versioning discipline (semver, changelog present?)
- Compatibility matrix with current Claude Code version

5. Banking-specific governance

- ZDR-addendum-compatible (no plugin telemetry back to Anthropic without ZDR)?
- Auditable: does plugin output land in audit log if needed?
- RBAC-enforceable through managed marketplace?
- Can be allow-listed centrally (Enterprise plan + managed plugin marketplace)?

Permissions audit - what to look for in plugin.json

```
{
  "permissions": {
    "tools": ["Read", "Write", "Bash", "Edit", ...],
    "mcp_servers": [...],
    "hooks": ["PreToolUse", "PostToolUse", ...]
  }
}
```

Red flags:

- Bash permission without a clear scoped need
- Network / WebFetch without explicit purpose
- PreToolUse hook on broad event match (* instead of specific tool name)
- MCP server registration with no public schema
- Unsigned or no checksum on releases

Yellow flags:

- Requests Write on the whole repo (should usually be path-scoped via CLAUDE.md)
- Subagents with the same permissions as the lead (no privilege separation)

Update cadence policy

Once installed, who tracks when to update?

Plugin tier | Update review | Owner

--- | --- | ---

Anthropic first-party (hookify, security-guidance, pr-review-toolkit, superpowers) | Monthly; major release notes review | Tech lead per team

Community / third-party | Quarterly; full re-vet on major version bump | Architect + tech lead

Internal plugins | Per change; treated as code | PR review

Pin versions in production environments. Don't auto-update on workstations either - coordinate quarterly.

MCP server specifics

MCP servers are remote - they receive your prompts and context. Additional checks:

- Endpoint TLS: HTTPS-only; certificate pinning where supported
- Auth model: OAuth (preferred), API key (acceptable with rotation), or anonymous (NEVER for client work)
- Data residency: For EU clients, EU-hosted MCP servers only - or explicit ZDR + data residency clause
- Logging: What does the MCP server log? For how long? GDPR data subject request flow?
- SLA: What's the uptime target? What's the failover plan?

Common safer MCPs for ITSS: Atlassian Rovo MCP Server (GA Feb 2026; covers Jira + Confluence + JSM in one), GitHub MCP, and a maintained Postgres MCP - vetted internally.

Postgres MCP - important caveat: the original Anthropic reference Postgres MCP server has been deprecated/archived. A SQL injection vulnerability was found in it by Datadog Security Labs (see <https://securitylabs.datadoghq.com/articles/mcp-vulnerability-case-study-SQL-injection-in-the-postgresql-mcp-server/>). Use a maintained community alternative, vet it through this checklist, and lock down credentials: read-only role for all dev sessions, scoped to specific schemas, rotated quarterly, fully audited. Production DB credentials should be categorically denied at the sandbox network.allowedDomains level - the agent's MCP should never touch prod from a dev laptop.

Incident playbook (when a plugin behaves badly)

1. Immediately: /plugin uninstall <name> on all affected machines (push policy through managed marketplace if Enterprise)
2. Within 1h: identify scope - which repos, which sessions, what was leaked/modified?
3. Within 24h: audit log review; check git history for unexpected commits
4. Within 1 week: publish internal post-mortem; update this checklist with the new red flag
5. Within 30 days: external disclosure if customer data was affected (GDPR Art. 33)

Anti-patterns

- ? Installing a plugin because it's popular (popularity ? safety)
- ? "We'll review the permissions later" - review BEFORE install
- ? Single-person decision to add to the team-wide marketplace
- ? No expiry on community plugins (re-vet quarterly, not "once and forget")
- ? Treating MCP servers as equivalent to local libraries - they're remote services

ITSS Workshop - 28 May 2026. Run this checklist quarterly + on every new install.